

INTERNSHIP PROJECT REPORT

GeoPython for Aerospace

Aircraft Route Analyzer — Indian Domestic Aviation

Submitted By
Organisation

Hritikesh Gaikwad, Utkarsh Chorode, Hriiday Garud
Rotten Grapes Private Limited

Table of Contents

Table of Contents.....	1
1. Abstract.....	2
2. Introduction	3
2.1 Background	3
2.2 Problem Statement.....	3
2.3 Objectives.....	3
2.4 Scope	4
3. Technology Stack	4
4. Data Sources	4
4.1 Airport Data	5
4.2 Route and Airline Data.....	5
5. Methodology	6
5.1 Haversine Distance Formula.....	6
5.2 Graph Construction	6
5.3 Shortest Path Algorithms	7
5.3.1 Dijkstra's Algorithm.....	7
5.3.2 A* Search Algorithm.....	7
5.4 Fuel and Emissions Model.....	7
6. System Architecture	8

6.1 Data Flow	8
7. Results.....	8
7.1 Distance Validation.....	9
7.2 Algorithm Benchmarking.....	9
7.3 Sample Route Results	9
8. Interactive Map Features.....	10
8.1 User Interface.....	10
8.2 Map Layers.....	10
8.3 Technical Implementation	11
9. GIS Exports.....	11
9.1 GeoJSON.....	11
9.2 Shapefile	11
10. Testing and Validation.....	11
11. Conclusion	12
11.1 Future Scope.....	13

1. Abstract

This report describes the design, development, and results of an Aircraft Route Analyzer created during a technology internship at Rotten Grapes Private Limited. The project, called **GeoPython for Aerospace – Aircraft Route Analyzer**, focuses on using Python and geospatial analysis techniques to study airline routes in India. The system was developed using data from 15 major Indian airports and 36 domestic flight routes operated by IndiGo, Air India, Akasa Air, and SpiceJet.

The project uses the Haversine Formula to calculate the distance between airports, while Dijkstra's Algorithm and A* Search Algorithm are used to find the shortest flight routes. A fuel consumption and CO₂ emission model was also developed using aircraft-specific data based on ICAO standards. The main output of the project is an interactive HTML map that allows users to view airline routes, filter routes by airline, find the shortest path between airports, and see estimated fuel usage for each route segment. The system also supports GeoJSON and Shapefile exports for GIS applications and includes a distance matrix for all airport pairs.

The results show that the Haversine Formula provides highly accurate distance calculations, with values very close to actual airline route distances. Both Dijkstra's and A* algorithms successfully found the same optimal routes in all test cases. The fuel consumption and emission estimates were also found to be within the expected ICAO benchmark ranges. Overall, the project demonstrates how Python-based geospatial tools can be used to analyse airline networks and support route planning in the aerospace sector.

2. Introduction

2.1 Background

India has one of the largest domestic aviation networks in the world, with millions of passengers travelling by air every year. Airlines such as IndiGo, Air India, Akasa Air, and SpiceJet are continuously expanding their services, while government initiatives like the UDAN scheme are improving air connectivity between major cities and smaller towns. Because of this growth, route planning, fuel efficiency, and environmental impact have become important areas of study in the aviation sector.

Python-based geospatial tools such as NetworkX, Folium, GeoPandas, and Shapely provide useful methods for analysing transportation networks and geographical data. These tools help in visualising routes, calculating distances, and finding the most efficient paths between locations. In this project, these technologies are applied to the field of civil aviation to analyse airline routes and better understand the structure of India's domestic air transport network.

2.2 Problem Statement

Given a network of Indian domestic airports and airline route data, the project addresses the following problems:

- Computing accurate great-circle distances between all airport pairs.
- Finding the shortest route (minimum distance) between any two airports using graph algorithms.
- Estimating fuel consumption and CO₂ emissions for each route based on aircraft type.
- Visualising the network interactively in a web browser with airline-specific filtering.
- Exporting the network data in standard GIS formats for further analysis.

2.3 Objectives

The specific objectives of this project are:

- To model 15 Indian airports and 36 domestic routes as a weighted undirected graph.
- To implement and compare Dijkstra's algorithm and A* search for route optimisation.
- To build an aircraft fuel and CO₂ model based on published performance parameters.
- To create a fully self-contained interactive HTML map with airline filtering.
- To export results in CSV, GeoJSON, and Shapefile formats.
- To validate all components through systematic algorithmic testing.

2.4 Scope

The project covers 15 major Indian airports, 36 domestic route pairs, and 4 commercial airlines operating during Q1 2026. International routes, cargo operations, and charter flights are outside the scope. All distance calculations use the WGS-84 spherical approximation (Haversine). Fuel models are based on publicly available aircraft performance manuals and ICAO standards.

3. Technology Stack

The project was developed entirely in Python 3.11 on Windows, using open-source libraries throughout. The following table summarises the technology stack:

Library / Tool	Purpose
Python	Core programming language
NetworkX	Graph construction, Dijkstra, A* pathfinding
Folium	Interactive Leaflet.js map generation
Pandas	CSV data loading, manipulation, export
GeoPandas	Geospatial data handling, Shapefile export
Shapely	Geometry creation (Point, LineString)
Math (stdlib)	Haversine formula trigonometry
JSON (stdlib)	Embedding path data into HTML
OS (stdlib)	File path management

Table 1: Technology stack used in the Aircraft Route Analyzer

4. Data Sources

4.1 Airport Data

Airport coordinates and IATA codes were compiled from publicly available DGCA publications and cross-validated against the OpenFlights airport database (openflights.org). The dataset covers 15 airports as listed in Table 2.

IATA	City	Latitude	Longitude	Airport Name
DEL	New Delhi	28.6139	77.2090	Indira Gandhi Intl
BOM	Mumbai	19.0896	72.8656	Chhatrapati Shivaji Maharaj
BLR	Bengaluru	13.1986	77.7066	Kempegowda Intl
HYD	Hyderabad	17.2403	78.4294	Rajiv Gandhi Intl
MAA	Chennai	12.9941	80.1709	Chennai Intl
CCU	Kolkata	22.6520	88.4463	Netaji Subhas Chandra Bose
PNQ	Pune	18.5822	73.9197	Pune Airport
AMD	Ahmedabad	23.0772	72.6347	Sardar Vallabhbhai Patel
JAI	Jaipur	26.8242	75.8122	Jaipur Intl
LKO	Lucknow	26.7606	80.8893	Chaudhary Charan Singh
GOI	Goa	15.3808	73.8314	Goa Intl (Dabolim)
COK	Kochi	10.1520	76.3919	Cochin Intl
BBI	Bhubaneswar	20.2444	85.8178	Biju Patnaik Intl
IXC	Chandigarh	30.6735	76.7885	Shaheed Bhagat Singh Intl
IDR	Indore	22.7218	75.8011	Devi Ahilya Bai Holkar

Table 2: 15 Indian airports used in the route network

4.2 Route and Airline Data

Route frequency data was assembled from DGCA Monthly Domestic Traffic Statistics (March 2026) and cross-referenced with airline timetable publications. The dataset covers 36 unique route pairs operated by the four airlines listed below:

Airline	Market Share	Aircraft Types	Route Count
IndiGo	~63%	A320neo, A321neo, ATR 72	23
Air India	~26%	A320neo, A321neo	20
Akasa Air	~5%	B737 MAX 8	10
SpiceJet	~4%	B737-800	9

Table 3: Airline market share and route coverage (DGCA March 2026)

5. Methodology

5.1 Haversine Distance Formula

The great-circle distance between two geographic coordinates is computed using the Haversine formula, which models the Earth as a perfect sphere of radius $R = 6,371$ km. This is the standard formula used in aviation for short to medium-range distance estimation.

The formula is defined as:

$$d = 2R \times \arcsin\left(\sqrt{\sin^2(\Delta\phi/2) + \cos(\phi_1) \times \cos(\phi_2) \times \sin^2(\Delta\lambda/2)}\right)$$

Where ϕ represents latitude, λ represents longitude (both in radians), and $\Delta\phi$, $\Delta\lambda$ represent the differences. For Indian domestic sectors (distances $< 3,000$ km), the Haversine model achieves accuracy within 0.3%–0.8% compared to the WGS-84 ellipsoidal model, making it entirely suitable for this application.

The implementation computes distances for all $C(15,2) = 105$ airport pairs and stores the results in `distance_matrix.csv`. Each pair also records the geodesic distance (computed using GeoPandas) for comparison.

5.2 Graph Construction

The 15 airports are modelled as nodes in an undirected weighted graph using the NetworkX library. Edges are added for each unique route pair present in the `airline_routes.csv` dataset, with the Haversine distance as the edge weight. The graph has the following properties:

- Nodes: 15 (one per airport)
- Edges: approximately 36 unique route pairs
- Graph type: undirected, weighted
- Edge weight unit: kilometres (Haversine distance)

- Connectivity: fully connected (all airports reachable from any other airport)

5.3 Shortest Path Algorithms

5.3.1 Dijkstra's Algorithm

Dijkstra's algorithm is a greedy graph search algorithm that finds the shortest path from a source node to all other nodes in a non-negative weighted graph. It maintains a priority queue (min-heap) of nodes ordered by their tentative distances and relaxes edges iteratively.

Time complexity: $O((V + E) \log V)$, where V is the number of nodes and E is the number of edges. For the 15-node graph used in this project, Dijkstra executes in under 1 millisecond per query.

5.3.2 A* Search Algorithm

A* extends Dijkstra's algorithm by incorporating a heuristic function $h(n)$ that estimates the remaining cost from any node n to the destination. This allows the algorithm to prioritise nodes that appear to be on the most promising path, reducing the number of nodes explored.

In this project, the heuristic function is the Haversine distance from the current airport to the destination airport. This heuristic is admissible (never overestimates the true cost) and consistent (satisfies the triangle inequality), guaranteeing that A* returns the optimal path.

Both algorithms are verified to produce identical shortest paths on all tested origin-destination pairs, confirming the correctness of both implementations.

5.4 Fuel and Emissions Model

The fuel model assigns an aircraft type to each route based on its distance and the operating airline, then computes total fuel burn, CO₂ emissions, and fuel cost. The model parameters are derived from Airbus and Boeing published performance data and ICAO Annex 16, Volume II.

Aircraft	Fuel (L/km)	Taxi (L)	Climb (L)	Seats	Typical Range
ATR 72-600	1.8	80	150	70	< 300 km
A320neo	3.2	200	700	180	300–1,200 km
A321neo	3.8	220	800	220	> 1,200 km
B737-800	3.4	210	720	189	SpiceJet routes
B737 MAX 8	3.0	200	680	189	Akasa Air routes

Table 4: Aircraft fuel model parameters (ICAO Annex 16 / Airbus-Boeing performance data)

CO₂ emissions are computed using the ICAO standard emission factor of 3.16 kg CO₂ per kg of Jet-A fuel burned, combined with a fuel density of 0.8 kg/litre. Per-passenger emissions assume an 85% load factor (consistent with DGCA published averages). Indian Aviation Turbine Fuel (ATF) price is taken as ₹90 per litre.

6. System Architecture

The project is structured as a collection of independent Python scripts, each responsible for a specific pipeline stage. All scripts read from the data/ directory and write outputs to the output/ directory.

Script	Purpose
distance_matrix.py	Reads airports.csv, computes Haversine + geodesic for all 105 pairs, exports distance_matrix.csv
fuel_model.py	Assigns aircraft types, calculates fuel burn, CO ₂ , and cost for each route, exports fuel_model.csv
shapefile_export.py	Creates GeoJSON and Shapefile exports of airports (Point) and routes (LineString)
interactive_map_v5.py	Builds the Folium base map, precomputes all Dijkstra paths, injects the dark UI panel, saves the HTML
testing.py	Runs 9 validation and benchmarking tests, saves 3 CSV result files to output/

Table 5: Project script architecture

6.1 Data Flow

- airports.csv and airline_routes.csv are the primary inputs.
- distance_matrix.py and fuel_model.py produce intermediate analysis CSVs.
- shapefile_export.py produces GIS-compatible files for QGIS / ArcGIS.
- interactive_map_v5.py is the main deliverable — it consumes all data and produces the self-contained HTML file.
- testing.py validates correctness and documents performance benchmarks.

7. Results

7.1 Distance Validation

The Haversine distances computed by the system were validated against published route distances from DGCA and airline sources. The table below shows the comparison for 10 key routes:

Route	Airports	Computed (km)	Published (km)
DEL–BOM	New Delhi – Mumbai	1,148	1,138
DEL–BLR	New Delhi – Bengaluru	1,741	1,740
DEL–CCU	New Delhi – Kolkata	1,304	1,305
DEL–MAA	New Delhi – Chennai	2,192	2,180
BOM–BLR	Mumbai – Bengaluru	842	845
BOM–MAA	Mumbai – Chennai	1,035	1,031
BOM–CCU	Mumbai – Kolkata	1,656	1,660
BLR–HYD	Bengaluru – Hyderabad	505	500
DEL–HYD	New Delhi – Hyderabad	1,258	1,260
CCU–BBI	Kolkata – Bhubaneswar	417	419

Table 6: Haversine distance validation against published airline data

All computed distances fall within 1.0% of published values, well below the 5% threshold set in the testing specification. The small differences (0.1%–0.7%) arise because published distances use the WGS-84 ellipsoid model while Haversine uses a spherical approximation.

7.2 Algorithm Benchmarking

Both Dijkstra's algorithm and A* search were benchmarked over 1,000 runs per route pair. On the 15-node sparse graph, both algorithms execute in under 5 milliseconds total for 1,000 runs, demonstrating near-identical performance at this scale. Both algorithms produced identical optimal paths for all 8 tested origin-destination pairs, confirming algorithmic correctness.

The A* advantage becomes significant on large-scale graphs (1,000+ nodes) due to the heuristic guiding the search. For production-scale route networks (e.g., all 150+ Indian airports), A* would explore significantly fewer nodes than Dijkstra.

7.3 Sample Route Results

The table below shows sample Dijkstra shortest-path results for selected routes, including fuel and emissions data:

Route	Distance	Aircraft	Fuel (L)	CO ₂ / pax (kg)
DEL–BOM	1,148 km	A320neo	4,574	135.6
BOM–BLR	842 km	A320neo	3,594	106.6
DEL–COK	2,840 km	A321neo	12,392	290.8
PNQ–BOM	118 km	ATR 72-600	442	30.8
IXC–MAA	2,154 km (via DEL)	A320neo	7,793	231.1

Table 7: Sample route results with fuel and emissions data

8. Interactive Map Features

The primary deliverable of this project is a fully self-contained interactive HTML file (interactive_route_map.html) that requires no server, no internet connection, and no additional software to run — only a web browser.

8.1 User Interface

The map features a dark aviation-themed control panel with the following interactive elements:

- Airline Selector — filter the visible route network by IndiGo, Air India, Akasa Air, SpiceJet, or all airlines combined.
- From / To Airport Selectors — choose any two of the 15 airports as origin and destination.
- Find Route Button — executes the precomputed Dijkstra path and renders it on the map with animated coloured polylines.
- Leg-by-Leg Result Card — displays total distance, flight time estimate, fuel consumption, CO₂ emissions (total and per passenger), fuel cost, and a breakdown of each individual flight leg with aircraft type and estimated time.

8.2 Map Layers

- Base map: CartoDB Dark Matter — a dark monochrome tile layer matching the UI theme.
- Background routes: all 36 route pairs displayed as faint blue lines.
- Active route: the selected shortest-path route displayed as a bright coloured polyline (colour-coded by airline).

- Airport markers: FontAwesome plane icons at each of the 15 airport locations with hover tooltips.
- Stop markers: colour-coded circles — green for origin, red for destination, orange for intermediate stops.

8.3 Technical Implementation

All shortest paths are precomputed at HTML generation time and embedded as a JSON object inside the JavaScript. This means route lookups in the browser are $O(1)$ dictionary lookups, making the UI respond instantly regardless of graph complexity. The entire application is embedded in a single .html file of approximately 430 KB.

9. GIS Exports

In addition to the interactive HTML map, the project exports the airport and route data in standard GIS formats for use in desktop GIS software such as QGIS or ArcGIS.

9.1 GeoJSON

- airports_points.geojson — 15 Point features, each with IATA code, city, latitude, and longitude attributes.
- routes_lines.geojson — 36 LineString features, each with origin IATA, destination IATA, distance, and frequency attributes.

9.2 Shapefile

The Shapefile export produces two complete sets of companion files (5 files each):

- airports.shp, .dbf, .prj, .shx, .cpg — airport point geometries with attribute table.
- routes.shp, .dbf, .prj, .shx, .cpg — route line geometries with attribute table.

All exports use the WGS-84 coordinate reference system (EPSG:4326), which is the standard for global geographic data.

10. Testing and Validation

A comprehensive testing module (testing.py) was developed to validate all project components. The module implements 9 independent tests and exports results to CSV files in the output/ directory.

#	Test Name	Description
1	Graph Structure Validation	Verifies 15 nodes, correct edges, and full graph connectivity
2	Distance Validation	Compares Haversine distances against 10 published route values; pass threshold < 5% error
3	Dijkstra vs A* Agreement	Confirms both algorithms produce identical paths for 8 route pairs
4	Algorithm Benchmarking	Times both algorithms over 1,000 runs each on 5 routes
5	Fuel Model Sanity Check	Validates fuel estimates against ICAO/Airbus benchmark ranges
6	Distance Matrix Completeness	Checks 105 rows, required columns, and zero null values
7	Shapefile Existence Check	Confirms all 10 Shapefile companion files are present
8	Airline Distribution Analysis	Computes market share percentages from airline_routes.csv
9	Hub Connectivity Analysis	Ranks all 15 airports by number of direct connections

Table 8: Testing module — 9 validation tests

11. Conclusion

This project successfully demonstrated the application of Python geospatial tools to a real-world aviation route analysis problem. The Aircraft Route Analyzer models 15 Indian airports and 36 domestic routes, implements two shortest-path algorithms, produces a multi-aircraft fuel and emissions model, and delivers a fully functional interactive map as a self-contained HTML file.

Key achievements include:

- Haversine distance computation achieving < 1% error against published airline data.
- Dijkstra and A* algorithms verified to produce identical optimal paths on all tested pairs.
- Fuel model estimates validated against ICAO Annex 16 benchmark ranges for all sectors.

- A fully self-contained interactive HTML map with dark aviation UI, airline filtering, and real-time route results requiring no server or external dependencies.
- Complete GIS export in GeoJSON and Shapefile formats for compatibility with QGIS and ArcGIS.
- A 9-test validation module providing documented proof of correctness.

The project demonstrates that open-source Python tools (NetworkX, Folium, GeoPandas, Pandas) are fully capable of producing production-quality geospatial analysis and visualisation systems for civil aviation applications. The methodology developed here can be extended to cover all 150+ DGCA-registered Indian airports, international routes, real-time NOTAM data, or weather-adjusted routing.

11.1 Future Scope

- Expand the airport network to cover all DGCA-registered airports (150+).
- Integrate real-time flight data via the ADS-B Exchange or FlightAware API.
- Add weather-adjusted routing using wind speed and direction data.
- Implement carbon offset cost calculation for airline sustainability reporting.
- Build a web API version using FastAPI for integration with airline systems.